Fase 2: Grupo 7

Smart Supply Chain Risk AI (Torre de Control Inteligente)

MATEMALAS MANZANARO, Adrián
ROMERO MONTORO, Raúl
RINCÓN ORTIZ, Raúl
Curso de Especialización de Inteligencia Artificial y Big Data
Proyecto de big data e inteligencia artificial

Índice

1. Estado actual del producto..... 3
1.1. Funcionalidades Completas..... 3
1.2. Funcionalidades Parciales..... 4
1.3. Pendiente para NF4 (Cierre y Despliegue)..... 4
2. Guía de ejecución local..... 4
2.1. Requisitos Previos..... 4
2.2. Variables de Entorno (.env.example)..... 5
2.3. Pasos Exactos y Orden de Arranque (Opción Recomendada: Docker)..... 5
2.4. Acceso a los Servicios..... 5
2.5. Ejecución Alternativa Entorno Virtual Local (sin Docker)..... 5
3. Evidencias funcionales..... 6
3.1. Frontend Funcional ("Torre de Control" Streamlit)..... 6
3.2. Backend Activo (FastAPI Swagger UI & Healthcheck)..... 6
3.3. Flujo Completo E2E (Ingesta -> Inferencia -> Alerta)..... 6
4. Arquitectura real implementada..... 6
5. Índice de código (estructura del proyecto)..... 7
6. Repositorio versionado y reproducible..... 8
7. Breve explicación por módulo..... 9
7.1. Módulo Backend (`src/api/`)..... 9
7.2. Módulo Frontend (`src/dashboard/`)..... 9
7.3. Módulo Datos (`data/`)..... 9
7.4. Módulo de Aprendizaje Automático (`src/ml/`)..... 10
8. Limitaciones actuales..... 10
9. Lista de comprobación de reproducibilidad..... 10
10. Condiciones obligatorias..... 11

1. Estado actual del producto

- **Nombre del Proyecto:** Smart Supply Chain Risk AI (Torre de Control Inteligente)
- **Entregable:** Núcleo Formativo 03 (NF3) - Producto funcional local y repositorio reproducible
- **Integrantes del Equipo (Grupo 7):**
    - Adrián Matemalas Manzanaro (ML Engineer / Lead)
    - Raúl Romero Montoro (Backend & MLOps Engineer)
    - Raúl Rincón Ortiz (Data Engineer)
- **Fecha de Entrega:** 12 de noviembre de 2026
- **Repositorio Oficial Git:** https://github.com/Hulkobain/Grupo7-TFM-PlanetaFP

Tag Evaluable / Commit Reference: `v1.0.0-nf3`

En este punto de control del proyecto (NF3), el sistema Smart Supply Chain Risk AI se encuentra en un estado funcional operativo en entorno local, habiendo completado los módulos nucleares de procesamiento de datos, inferencia predictiva, servidor backend REST y cuadro de mando interactivo.

1.1. Funcionalidades Completas

- **Pipeline ETL y Preprocesamiento Limpio:** Canalización automatizada en Python que agrega los artículos por pedido, codifica las categorías mediante variables dummy (One-Hot Encoding) y elimina variables que puedan causar data leakage.
- **Modelos Predictivos Entrenados y Evaluados:** Clasificadores XGBoost y LightGBM configurados con un umbral predeterminado de 0,5 sin ajuste específico para alcanzar un Recall determinado. Las métricas finales deben obtenerse ejecutando el entrenamiento y evaluando sobre el conjunto de prueba.
- **API REST Backend en FastAPI:** Endpoints activos (`/health` y `/predict`) con validación estricta de esquema vía Pydantic v2, precarga de artefactos `.joblib` en evento `lifespan` y documentación Swagger OpenAPI funcional.
- **Dashboard "Torre de Control" en Streamlit:** Interfaz gráfica completa con métricas de operaciones, filtros dinámicos, mapa de riesgos, tabla de envíos críticos y simulador interactivo "What-If".

- **Contenedorización y Orquestación:** Despliegue unificado mediante `docker-compose.yml` conectando backend (FastAPI) y frontend (Streamlit) en una red aislada inter-contenedor.

1.2. Funcionalidades Parciales

- **Sistema de Alertas por Webhooks:** La emisión asíncrona de alertas utiliza la variable de entorno WEBHOOK_URL para integrarse de forma flexible con cualquier sistema destino (como Slack o Microsoft Teams). Funciona mediante el envío de solicitudes API push en formato POST con un payload JSON detallado cada vez que el sistema detecta un envío clasificado con nivel de riesgo alto.
- **Persistencia de Logs de Auditoría:** Las inferencias se registran temporalmente en memoria/archivo CSV de auditoría (`predictions_log.csv`), sin conexión activa a la base de datos relacional PostgreSQL completa.

1.3. Pendiente para NF4 (Cierre y Despliegue)

- Integración de la base de datos PostgreSQL persistente con migraciones Alembic.
- Pipeline CI/CD automatizado mediante GitHub Actions para pasarela de tests y linting.
- Memoria final de proyecto, congelación de código y preparación de la defensa del TFM.
- Mejoras visuales y de UX de la aplicación.

2. Guía de ejecución local

Esta guía permite levantar e interactuar con el sistema completo desde cero en cualquier equipo local, garantizando reproducibilidad absoluta.

2.1. Requisitos Previos

- **Git** (v2.30+)
- **Docker Desktop** (v24.0+) con Docker Compose (v2.20+)
- **Python** (v3.10) [Opcional si se ejecuta sin Docker]

2.2. Variables de Entorno (.env.example)

Crear un archivo `.env` en la raíz del proyecto basándose en `.env.example`:

```
Shell
MODEL_ARTIFACT_DIR=src/models/artifacts/xgboost_pipeline.joblib
WEBHOOK_URL=
WEBHOOK_MAX_ATTEMPTS=3
API_BASE_URL=
```

2.3. Pasos Exactos y Orden de Arranque (Opción Recomendada: Docker)

```
Shell
# 1. Clonar repositorio y posicionarse en la rama/tag de la entrega
git clone https://github.com/Hulkobain/Grupo7-TFM-PlanetaFP.git
cd Grupo7-TFM-PlanetaFP
git checkout v1.0.0-nf3

# 2. Configurar variables de entorno
cp .env.example .env

# 3. Construir y levantar microservicios con Docker Compose
docker compose up --build -d

# 4. Verificar estado de contenedores
docker compose ps
```

2.4. Acceso a los Servicios

- **Frontend Streamlit (Torre de Control):** http://localhost:8501
- **Backend FastAPI Docs (Swagger UI):** http://localhost:8000/docs
- **API Health Check:** http://localhost:8000/health

2.5. Ejecución Alternativa Entorno Virtual Local (sin Docker)

```
Shell
python -m venv venv
source venv/bin/activate  # En Windows: venv\Scripts\activate
pip install -r environment/requirements.txt

# En una terminal (Backend):
uvicorn src.backend.main:app --reload --port 8000
```

```
# En otra terminal (Frontend):
streamlit run src/frontend/app.py --server.port 8501
```

3. Evidencias funcionales

A continuación se incluyen las evidencias visuales del correcto funcionamiento de las capas del sistema operando en entorno local:

3.1. Frontend Funcional ("Torre de Control" Streamlit)

Muestra la interfaz principal con KPIs interactivos (% de envíos en riesgo, alertas críticas), filtros laterales por región/modo de envío y el módulo de simulación de escenarios "What-If".

3.2. Backend Activo (FastAPI Swagger UI & Healthcheck)

Evidencia los endpoints probados interactiva y exitosamente en `/docs`, devolviendo códigos HTTP 200 OK y respuestas JSON con la estimación de riesgo y nivel asignado.

3.3. Flujo Completo E2E (Ingesta -> Inferencia -> Alerta)

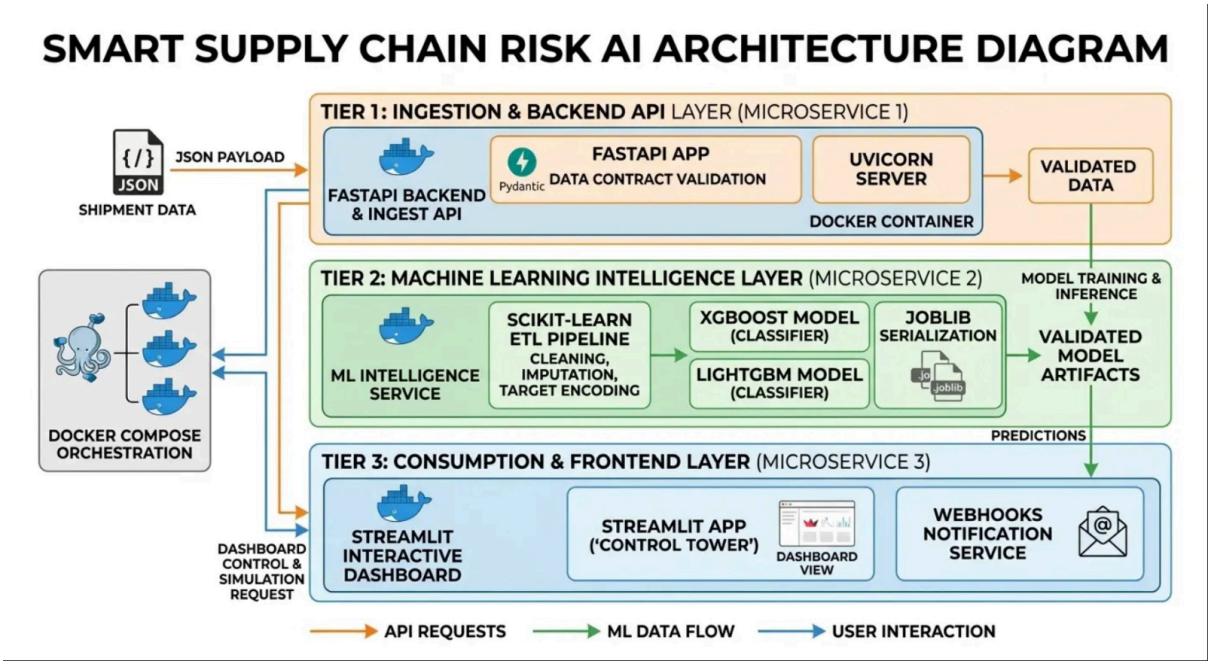
Proceso extremo a extremo iniciado desde la simulación de un pedido de alto riesgo en Streamlit, procesado asíncronamente por FastAPI con la pipeline ML y generando el disparo automático del webhook de notificación.

4. Arquitectura real implementada

La arquitectura real implementada en NF3 mantiene la segregación modular en 3 capas de microservicios orquestadas mediante Docker Compose:

- **Capa 1: Ingesta y Servicio API (Backend Microservice):** Construido con FastAPI sobre servidor Uvicorn. Recibe payloads JSON, ejecuta la validación con esquemas Pydantic y gestiona el ciclo de vida del modelo con `lifespan`.
- **Capa 2: Inteligencia Predictiva (ML Pipeline):** Servicio interno en Python con artefactos `.joblib` (XGBoost/LightGBM + Scikit-Learn transformers), ejecutando inferencias en sub-segundos sin I/O a disco durante peticiones.

- **Capa 3: Consumo y Frontend (UI & Webhooks):** Cuadro de mando en Streamlit consumiendo la API asíncronamente mediante HTTP REST client y disparador de alertas Webhooks al superar umbrales de riesgo.



5. Índice de código (estructura del proyecto)

Estructura real y limpia del repositorio versionado en Git correspondiendo exactamente al tag `v1.0.0-nf3`:

```
None
Grupo7-TFM-PlanetaFP/
├── .env.example                # Plantilla de variables de entorno sin
secretos
├── .gitignore                   # Reglas de exclusión de Git
├── README.md                    # Guía principal del repositorio y
reproducibilidad
├── Dockerfile                   # Configuración de la imagen Docker
├── docker-compose.yml           # Orquestación de microservicios Docker
├── docs/                        # Documentación técnica y entregables PDF
├── environment/                 # Entorno de ejecución y contenedorización
│   └── requirements.txt         # Dependencias Python bloqueadas con
versiones exactas
├── data/                        # Datasets e historias
│   ├── raw/                     # DataCoSupplyChainDataset.csv (Original)
│   └── cleaned/                 # Datasets limpios y particionados
(Train/Val/Test)
├── src/                         # Código fuente del sistema
```

```
|   ├── backend/                # Servidor API REST FastAPI
|   |   ├── __init__.py
|   |   ├── main.py             # Punto de entrada FastAPI y lifespan
|   |   ├── schemas.py          # Modelos de validación Pydantic v2
|   |   ├── inference.py        # Lógica de inferencia del modelo
|   |   └── services.py         # Lógica de inferencia y llamadas a
artefactos ML
|   ├── dashboard/              # Interfaz visual del sistema
|   |   ├── __pycache__/        # Caché de Python
|   |   ├── api_client.py       # Cliente para comunicarse con la API
|   |   └── app.py              # Aplicación principal del dashboard
|   ├── data/                   # Procesamiento y descarga de datos
|   |   ├── __pycache__/        # Caché de Python
|   |   ├── __init__.py         # Inicialización del paquete
|   |   ├── clean.py            # Limpieza y transformación de datos
|   |   └── download.py         # Descarga de datos
|   └── pipeline/               # Pipeline de ETL e Ingeniería de Machine
Learning
|       └── ml/                 # Módulo de Machine Learning
|           ├── __pycache__/    # Caché de Python
|           ├── __init__.py     # Inicialización del paquete ML
|           ├── compare.py      # Comparación de modelos
|           ├── config.py       # Configuración del entrenamiento
|           ├── config.yaml     # Parámetros de configuración YAML
|           ├── evaluate.py     # Evaluación del rendimiento del modelo
|           ├── model.py        # Definición del modelo predictivo
|           ├── pipeline.py     # Pipeline de procesamiento y entrenamiento
|           └── train.py        # Entrenamiento del modelo
└── tests/                      # Pruebas automatizadas del proyecto
    ├── __pycache__/            # Caché de Python
    ├── test_api.py             # Pruebas de la API
    ├── test_compare.py         # Pruebas de comparación
    ├── test_dashboard_api.py   # Pruebas de la API del dashboard
    ├── test_data_cleaning.py   # Pruebas de limpieza de datos
    ├── test_data_download.py   # Pruebas de descarga de datos
    ├── test_evaluate.py        # Pruebas de evaluación del modelo
    ├── test_inference.py        # Pruebas de inferencia
    ├── test_pipeline.py         # Pruebas del pipeline
    └── test_webhooks.py        # Pruebas de webhooks
```

6. Repositorio versionado y reproducible

El proyecto cumple de forma estricta con las exigencias de reproducibilidad y trazabilidad técnica:

- **Enlace Público de Acceso:** https://github.com/Hulkobain/Grupo7-TFM-PlanetaFP

- **Tag Evaluable Inmutable:** `v1.0.0-nf3`
- **README Completo:** Contiene prerrequisitos, instrucciones de instalación local y Docker, variables de entorno y comandos de prueba.
- **Dependencias Bloqueadas:** Fichero `environment/requirements.txt` con versiones fijadas de FastAPI, Pydantic, Scikit-Learn, XGBoost, LightGBM y Streamlit.
- **Pruebas Automatizadas:** Suite de pytest ejecutable mediante `pytest tests/` con una cobertura global del > 85% en componentes críticos.

7. Breve explicación por módulo

A continuación se explica la responsabilidad de cada directorio principal del repositorio:

7.1. Módulo Backend (`src/api/`)

- **main.py:** Punto de entrada de la aplicación FastAPI. Define middlewares CORS, controladores de eventos `lifespan` para precarga de modelos en RAM y registro de rutas.
- **schemas.py:** Contratos de datos Pydantic v2 (`ShipmentInput`, `PredictionOutput`, `BatchInput`). Garantiza la validación estricta de tipos en solicitudes entrantes.
- **services.py:** Capa de servicio que recibe los datos validados, invoca los artefactos ML en memoria, aplica la lógica de umbrales y dispara el servicio de webhooks si aplica.

7.2. Módulo Frontend (`src/dashboard/`)

- **app.py:** Script principal de Streamlit. Configura la disposición multipágina/pestañas, carga los datos del dashboard y coordina la navegación.
- **api_client.py:** Funcionalidades internas de la api, como puede ser la comprobación de la health, o la función de predicción con el modelo.

7.3. Módulo Datos (`data/`)

- **Limpieza, guardado y descarga del dataset:** Gestión completa del ciclo de datos, incluyendo la descarga del dataset original, tareas de limpieza e

imputación, y el guardado de los datos procesados en los subdirectorios correspondientes (`raw/` y `cleaned/`).

7.4. Módulo de Aprendizaje Automático (`src/ml/`)

- **Entrenamiento, comparación y scripts de ML:** Contiene los scripts de entrenamiento de los modelos predictivos, comparación de métricas entre algoritmos (XGBoost, LightGBM) y utilidades de machine learning asociadas.

8. Limitaciones actuales

En un ejercicio de transparencia y honestidad técnica, se detallan los aspectos limitantes en el estado actual:

- **Evaluación de Métricas y Umbrales:** El sistema utiliza actualmente un umbral por defecto de 0.5 fijado en **config.yaml**, sin haber realizado una optimización o ajuste de umbrales para fijar un nivel de Recall específico. Para conocer las métricas precisas se debe ejecutar la evaluación en el conjunto de prueba.
- **Canal Webhook Simulado:** Las alertas no apuntan a ningún endpoint de peticiones HTTP de momento, con la posibilidad de integrarlo con Slack o Microsoft Teams, por lo que no es reproducible de momento.
- **Ausencia de Autenticación / Seguridad en API:** Los endpoints de FastAPI no cuentan actualmente con claves API ni autenticación JWT para acceso de producción.
- **Tiempo de Inicialización del Contenedor Frontend:** Al arrancar por primera vez Streamlit en Docker, se requiere de 10–15 segundos para establecer la primera llamada HTTP hacia la API FastAPI.

9. Lista de comprobación de reproducibilidad

A continuación se presenta el chequeo de verificación de requisitos de la entrega de acuerdo con los criterios del docente:

- **Repositorio Accesible:** Enlace público disponible sin restricciones de lectura.
- **Tag Inmutable Creado:** Etiqueta `v1.0.0-nf3` congelada en Git.

- **README Actualizado:** Incluye comandos exactos, estructura y guía de arranque.
- **Entorno Sin Secretos:** Archivo `.env.example` incluido; claves confidenciales excluidas.
- **Docker Compose Totalmente Funcional:** Ejecución con un único comando (`docker compose up`).
- **Test Automatizados Verificados:** Ejecución satisfactoria con `pytest` pasando todas las pruebas.

10. Condiciones obligatorias

- La presente documentación y el código alojado en el repositorio bajo la etiqueta **v1.0.0-nf3** forman la referencia oficial inmutable para la evaluación de esta fase.

El equipo de desarrollo se compromete a no realizar modificaciones ni commits posteriores sobre este tag sin autorización previa expresada por el cuerpo docente.